

HA NOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

o0o



Distributed Query Optimization on Columnar Storage

Big Data and Processing

Class Code: IT5427E

Supervisor: Assoc. Prof. Nguyen Binh Minh

Members of our group:

Nguyễn Viết Minh	20252074M	minh.nv252074m@sis.hust.edu.vn
Trương Gia Bách	20252277M	bach.tg252277m@sis.hust.edu.vn
Ngô Việt Anh	20252570M	anh.nv252570m@sis.hust.edu.vn

Abstract

Modern big data systems increasingly store analytical datasets in distributed object storage and process them through engines such as Apache Spark. In this report, we study the project *Distributed Query Optimization on Columnar Storage*, implemented as a Spark and MinIO benchmark over NYC Yellow Taxi trip records. The project follows the large-scale trajectory-processing theme: each taxi trip is represented as a trajectory segment from an origin taxi zone to a destination taxi zone, with temporal, spatial, distance, fare, and payment attributes normalized into a common `trips_clean` schema.

The report analyzes the complete project pipeline: downloading and uploading raw taxi data, normalizing schema differences, preparing multiple benchmark layouts, executing Spark SQL workloads, and interpreting runtime, shuffle, spill, and physical-plan metrics. The benchmark evaluates four optimization dimensions: file format selection, temporal partitioning, spatial hash bucketing, and distributed join/skew handling. Parquet, ORC, and Avro are compared under scan, aggregation, route-ranking, and origin-destination workloads. Raw Parquet is compared with year/month partitioned Parquet, and a 16-bucket hash layout over `origin_zone_id` is evaluated for zone-targeted queries. Finally, SortMerge, broadcast, and salted joins are benchmarked using a taxi-zone dimension table.

The results show that file format and layout choices strongly affect distributed query performance. Parquet is approximately $9.8\times$ faster than Avro on average, while ORC is approximately $12.3\times$ faster. Year/month partition pruning reduces runtime by 20–30% for time-filtered queries, while hash bucketing improves zone-targeted lookup and join queries by about 43–44%. For join optimization, broadcasting the 265-row zone dimension reduces runtime from 5.27s for SortMerge join to 1.15s and eliminates the observed 743 MB disk spill. These findings confirm that scalable trajectory analytics depends not only on cluster resources, but also on choosing storage formats, data layouts, and join strategies that match the workload.

1 Introduction

Big Data and Processing systems are designed to handle data volumes that exceed the capacity of a single machine. In practice, such systems must support not only data storage, but also repeated analytical queries over large fact tables. These queries often involve projection, filtering, aggregation, joins, window functions, and ranking. If the physical layout of data is not aligned with the query workload, distributed processing can waste large amounts of time reading unnecessary columns, scanning irrelevant files, shuffling data across workers, or spilling intermediate results to disk.

The project analyzed in this report addresses the topic *Distributed Query Optimization on Columnar Storage*. The project uses NYC Yellow Taxi trip records from 2025 as a realistic trajectory dataset. Rather than treating each record only as a financial transaction, the project models each trip as a movement segment from `origin_zone_id` to `destination_zone_id` over time. This representation supports common urban analytics tasks such as route ranking, origin-destination volume analysis, zone-level revenue aggregation, peak-hour flow analysis, and spatial-skew investigation.

The project is implemented using Apache Spark as the distributed compute engine and MinIO as an S3-compatible object storage system. Data is read and written through the `s3a://` protocol. A set of Python scripts automates dataset download, upload to object

storage, schema normalization, benchmark dataset creation, query execution, and metric collection. A set of notebooks provides data-quality inspection, trajectory exploration, and visualization of benchmark results.

The main research question is: *How do storage format, data layout, and join strategy affect Spark SQL performance for large-scale trajectory analytics?* To answer this question, the project compares row-oriented Avro with columnar Parquet and ORC, evaluates time-based partition pruning, evaluates spatial hash bucketing, and compares multiple join strategies under a naturally skewed spatial key.

The key contributions of this report are:

1. **End-to-end project description.** We explain the data flow from raw NYC taxi files to normalized benchmark datasets on MinIO.
2. **Trajectory schema analysis.** We describe the common `trips_clean` data model used by notebooks and Spark SQL benchmarks.
3. **Storage-format evaluation.** We compare Parquet, ORC, and Avro using projection, aggregation, route-ranking, and origin–destination queries.
4. **Layout optimization evaluation.** We evaluate temporal partitioning and spatial hash bucketing as data-skipping techniques.
5. **Join/skew analysis.** We compare SortMerge, broadcast, and salted joins and explain their shuffle/spill behavior.

The remainder of this report is organized as follows. Section 2 reviews background knowledge. Section 3 defines the problem and objectives. Section 4 describes the system architecture. Section 5 presents the methodology. Section 6 details the experimental setup. Section 7 analyzes results. Section 8 discusses limitations, and Section 9 concludes the report.

2 Background and Related Work

2.1 Distributed Data Processing

Distributed data processing frameworks split a large job into many smaller tasks that run in parallel. MapReduce introduced a simple model for large-scale batch computation using map and reduce functions [1]. Apache Spark improved on this model by supporting in-memory computation and richer APIs such as DataFrames and SQL [2, 3]. Spark SQL uses the Catalyst optimizer to transform logical queries into physical plans and can automatically apply optimizations such as predicate pushdown, column pruning, join selection, and Adaptive Query Execution [4].

For big-data analytics, performance is not determined only by CPU speed. Query time also depends on how many bytes must be read from storage, how many files must be opened, how much data must be shuffled between workers, and whether intermediate data spills to disk. These factors motivate query optimization at both the logical and physical levels.

2.2 Columnar Storage

Traditional row-oriented formats store each record as a sequence of fields. This layout is efficient for transactional workloads that read or write entire records, but it is inefficient for analytical workloads that scan a few columns across many rows. Column-store database research showed that column-oriented storage can improve scan efficiency and compression for analytical workloads [5, 6].

Parquet and ORC are columnar file formats commonly used in Spark data lakes. They store data in column chunks and include metadata such as column statistics, null counts, and compression information. These features allow Spark to read only needed columns and sometimes skip row groups or stripes that cannot match a predicate. Avro, by contrast, is row-oriented. It is useful for schema evolution and event serialization, but it requires decoding full records even when a query needs only a subset of fields.

2.3 Partitioning and Bucketing

Partitioning organizes data files into directories based on column values. For example, a taxi dataset may be written as `trip_year=2025/trip_month=1`. When a query filters on those columns, Spark can prune irrelevant directories before scanning file contents. Partition pruning is especially useful for time-window analytics, because most analytical questions focus on a specific month, day, or hour.

Bucketing clusters records by a hash of a key. In this project, the key is `origin_zone_id`. A hash-bucketed layout writes records into buckets computed as `PMOD(HASH(origin_zone_id), 16)`. If a query targets a single origin zone and includes the corresponding bucket predicate, Spark can read one bucket instead of scanning all data. Bucketing is therefore useful for repeated spatial-key lookups and joins.

2.4 Join Strategy and Data Skew

Distributed joins are often expensive because they require data movement. SortMerge join repartitions both inputs by the join key, sorts each partition, and merges matching keys. This is robust when both inputs are large, but it can be wasteful when one table is very small. Broadcast join sends a small table to every executor and joins locally, eliminating the need to shuffle the large table.

Data skew occurs when some keys appear much more frequently than others. In taxi trajectory data, airports and dense business districts may produce many more trips than ordinary zones. If a skewed key is used in a join, one worker may receive far more rows than others, causing stragglers and spill. Salted joins mitigate skew by adding a secondary salt key, distributing a hot key over multiple partitions. However, salting adds overhead and is not always beneficial, especially when broadcast join is possible.

3 Problem Statement

The project problem is to analyze performance bottlenecks in large-scale distributed analytical queries and propose optimization strategies using columnar storage and workload-aware data layouts. The selected application is large-scale taxi trajectory processing using NYC Yellow Taxi data.

The project requirements are mapped into four concrete goals:

- **Format comparison.** Compare Parquet, ORC, and Avro for trajectory scan and aggregation workloads.
- **Partition optimization.** Evaluate whether partitioning by `trip_year` and `trip_month` improves time-window queries.
- **Bucketing optimization.** Evaluate whether hash bucketing by `origin_zone_id` improves zone-targeted lookup and join queries.
- **Join/skew optimization.** Compare SortMerge, broadcast, and salted joins when joining trajectory records with the `taxi_zone_dim` table.

The workload is designed around real analytical questions: How many trips occurred in a time window? Which origin zones are most active? Which routes are most common? Which boroughs and zones generate high revenue? How much do layout and join strategy affect runtime?

4 System Architecture

4.1 High-level Architecture

The project uses a Spark-on-object-storage architecture. Spark is the compute layer and MinIO is the storage layer. Raw NYC taxi files are downloaded locally, uploaded to MinIO, normalized by Spark, and written back to MinIO in multiple benchmark layouts. Spark then reads these layouts through `s3a://` paths and executes SQL benchmarks.

The architecture has four main components:

- **Data source:** NYC Yellow Taxi monthly Parquet files for 2025.
- **Object storage:** MinIO bucket `taxi-data`, used like an S3-compatible data lake.
- **Compute engine:** Apache Spark cluster with a master and workers.
- **Experiment layer:** Python scripts and notebooks for dataset preparation, query execution, visualization, and analysis.

The documented cluster configuration uses one physical node with four Spark workers. Each worker has 12 cores and 24 GB of memory, resulting in 48 cores and 96 GB of worker memory. The project also includes Docker Compose files for optional multi-node deployment.

4.2 Data Flow

The data pipeline follows these steps:

1. **Download data:** `install_dataset.py` downloads monthly NYC Yellow Taxi Parquet files.
2. **Upload to MinIO:** `upload_to_minio.py` uploads local Parquet files to the MinIO bucket.

3. **Normalize schema:** `trajectory_utils.py` maps raw taxi columns into the common `trips_clean` schema.
4. **Prepare benchmark layouts:** `prepare_benchmark_datasets.py` writes raw Parquet, raw ORC, raw Avro, partitioned Parquet, bucketed Parquet, and reference dimensions.
5. **Run benchmarks:** `run_benchmark.py` executes SQL query groups and records runtime, shuffle, spill, rows, tasks, and physical-plan evidence.
6. **Visualize and interpret:** notebooks summarize schema, data quality, trajectory patterns, and benchmark results.

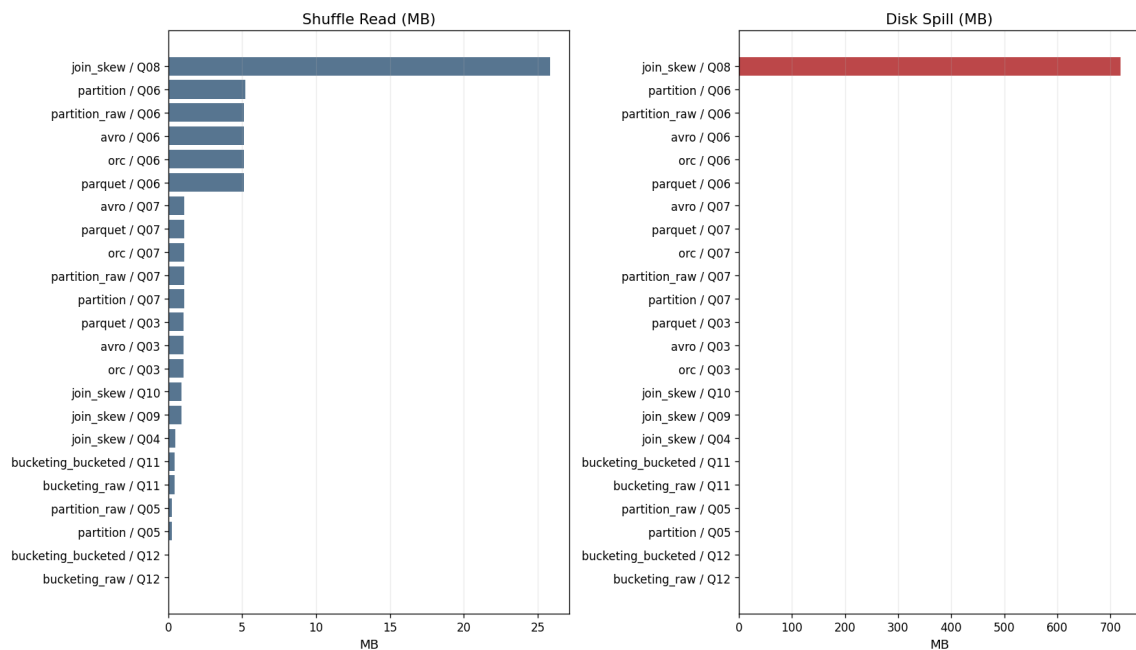


Figure 1: Shuffle and spill visualization from the benchmark notebook. These metrics identify expensive distributed execution stages.

4.3 Repository Structure

The repository is organized into documentation, notebooks, source scripts, and result folders:

- **docs/:** project brief, setup instructions, data dictionary, benchmark protocol, and result summary.
- **notebooks/:** schema normalization, data-quality exploration, trajectory exploration, and benchmark visualization notebooks.
- **src/:** Python scripts for configuration, downloading, uploading, schema normalization, benchmark dataset preparation, and benchmark execution.
- **results/:** output folders for format, partition, bucketing, and join/skew benchmark results.

This structure makes the project reproducible: scripts build the datasets, notebooks explain the data, and result summaries connect measured runtime to physical query behavior.

5 Methodology

5.1 Trajectory Data Model

The project defines a common logical table called `trips_clean`. Each row represents one taxi trip as a trajectory segment:

trajectory segment = (origin zone, destination zone, pickup time, dropoff time).

This model is zone-based rather than raw GPS point-based. It is appropriate because modern NYC taxi records contain pickup and dropoff zone IDs. These zone IDs can be joined with `taxi_zone_dim` to obtain borough and zone names.

Table 1: Important fields in the normalized `trips_clean` schema.

Group	Fields	Purpose
Temporal	<code>pickup_ts</code> , <code>dropoff_ts</code> , <code>trip_date</code> , <code>trip_year</code> , <code>trip_month</code>	Time filtering and partition pruning
Spatial	<code>origin_zone_id</code> , <code>destination_zone_id</code> , <code>route_key</code>	Origin-destination trajectory analytics
Business	<code>vendor_name_norm</code> , <code>payment_type_norm</code> , <code>passenger_count</code>	Vendor, payment, and passenger analysis
Measures	<code>trip_distance</code> , <code>fare_amt</code> , <code>tip_amt</code> , <code>tolls_amt</code> , <code>total_amt</code>	Revenue and distance aggregation
Layout	<code>origin_zone_bucket</code>	Hash bucket used for spatial data skipping
Metadata	<code>trajectory_mode</code> , <code>schema_family</code>	Schema and trajectory-type description

The schema normalization notebook confirms that all 12 months in 2025 use the modern zone-based schema with 20 raw columns. Core normalized fields have full coverage, while `passenger_count` has a high null rate. Figure 2 shows the coverage of key normalized columns.

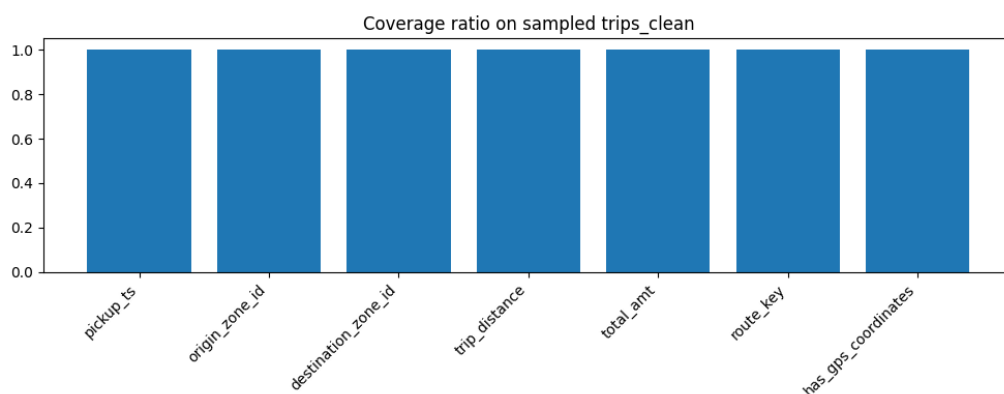


Figure 2: Coverage of important normalized trajectory columns. Core trajectory fields are available for all records.

5.2 Data Understanding

The benchmark uses approximately 48.7 million trips from 2025. Data-quality analysis checks null rates, zero-distance trips, negative fares, negative durations, payment anom-

lies, and self-loop trips. The project reports that key columns such as pickup time, dropoff time, origin zone, destination zone, trip distance, fare, total amount, and route key have complete coverage. Important anomalies are summarized in Table 2.

Table 2: Selected data-quality observations for the 2025 taxi dataset.

Observation	Count	Rate
Zero-distance trips	1,402,958	2.879%
Negative-fare trips	973,721	1.998%
Negative-duration trips	2,235	0.005%
Unknown payment type	11,611,894	23.833%
Self-loop trips	2,274,007	4.667%

These anomalies are important for data understanding, but they do not invalidate the query benchmark because the study focuses on execution behavior. Self-loop trips are valid intra-zone movements. Negative fares and extreme distances may affect business interpretation but do not change the storage-format or join-strategy conclusions.

The trajectory notebook also shows temporal and spatial skew. Average daily volume is approximately 133,487 trips, with counts ranging from 68,111 to 183,035. The top origin zone is Upper East Side South (zone ID 237), with 2,125,560 trips, about 4.4% of the dataset. This concentration motivates the join/skew benchmark.

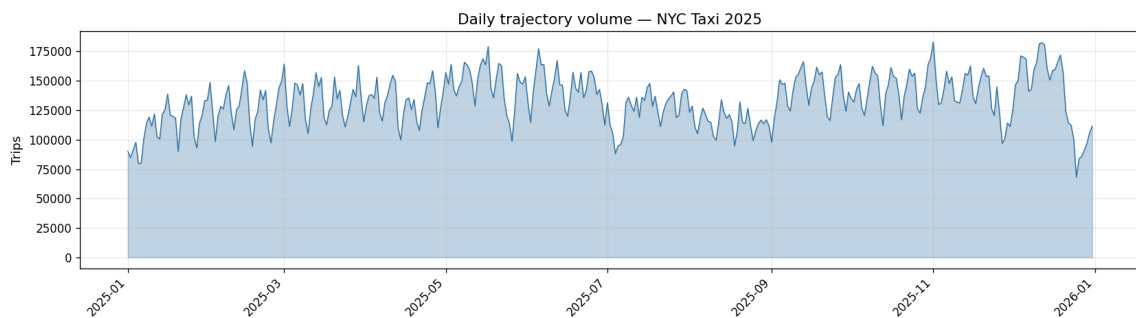


Figure 3: Daily taxi trip activity in 2025. The temporal pattern motivates time-window filtering and partition pruning.

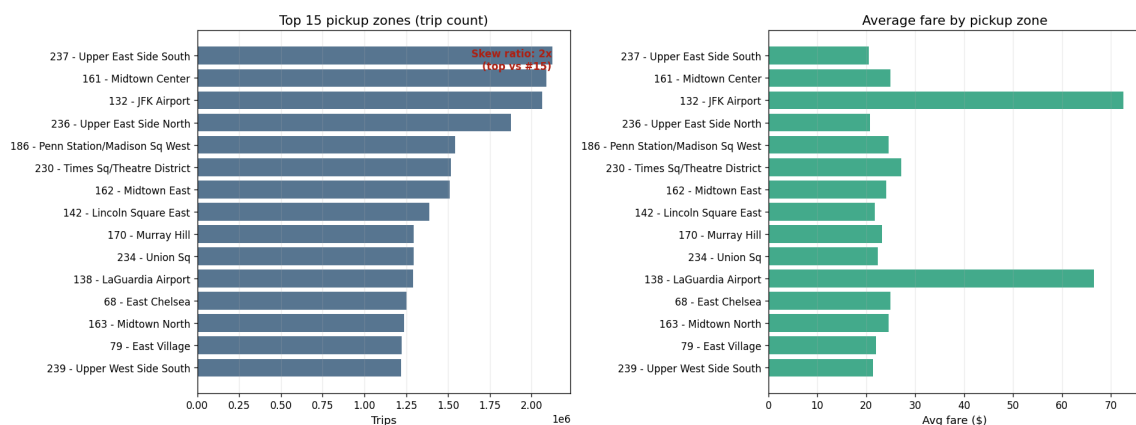


Figure 4: Top origin zones in the taxi dataset. Spatial concentration motivates bucketing and skew-aware join analysis.

5.3 Benchmark Dataset Layouts

The preparation script writes six logical benchmark datasets:

- `bench/parquet/raw`: normalized raw layout in Parquet.
- `bench/orc/raw`: normalized raw layout in ORC.
- `bench/avro/raw`: normalized raw layout in Avro.
- `bench/parquet/partitioned_year_month`: Parquet partitioned by `trip_year` and `trip_month`.
- `bench/parquet/bucketed_origin_zone_hash`: Parquet bucketed by hash of `origin_zone_id`.
- `bench/reference/taxi_zone_dim`: zone dimension table used for joins.

The hash bucket is computed as:

$$\text{origin_zone_bucket} = \text{PMOD}(\text{HASH}(\text{origin_zone_id}), 16).$$

The bucketed layout allows zone-targeted queries to include both `origin_zone_id` and `origin_zone_bucket`, reducing the amount of data scanned.

5.4 Benchmark Queries

The project defines 12 SQL benchmark queries. They are grouped by experiment type, as shown in Table 3.

Table 3: Benchmark query groups.

Group	Query IDs	Purpose
Format	Q01, Q03, Q06, Q07	Projection scan, vendor-zone aggregation, route ranking, OD volume
Partition	Q02, Q05, Q06, Q07	Time filter, daily zone flow, route ranking, monthly OD volume
Bucketing	Q11, Q12	Zone lookup and zone join using hash-bucketed layout
Join/skew	Q04, Q08, Q09, Q10	Skew aggregation, SortMerge join, broadcast join, salted join

The query set is representative of large-scale trajectory analytics because it includes projection, filtering, aggregation, grouping, ranking, and joins. Median runtime over three measured runs is used as the main runtime metric.

6 Experimental Setup

6.1 Hardware and Software

The documented benchmark runs on one physical node using Docker Compose. The Spark cluster contains one master and four workers. Each worker is configured with 12 cores and 24 GB memory, giving 48 total worker cores and 96 GB memory. MinIO provides S3-compatible object storage, and Spark accesses it through the Hadoop S3A connector.

The project software stack includes Python, PySpark 4.0.1, Spark SQL, MinIO, Docker Compose, and Jupyter notebooks. Spark packages include Hadoop AWS dependencies and the Spark Avro package. The project uses environment variables in `.env` and `project_config.py` to configure Spark master URL, driver host, MinIO endpoint, bucket name, access keys, result paths, and Spark package dependencies.

6.2 Benchmark Protocol

The benchmark protocol is:

1. Upload raw NYC taxi Parquet files to MinIO.
2. Use notebooks to understand schema, data quality, temporal patterns, route patterns, zone skew, and bucket distribution.
3. Prepare benchmark datasets in Parquet, ORC, Avro, partitioned Parquet, bucketed Parquet, and reference-dimension layouts.
4. Run query groups for format, partition, bucketing, and join/skew experiments.
5. Save CSV metrics and `EXPLAIN FORMATTED` plans.
6. Compare runtime, shuffle, spill, and physical-plan evidence.

The benchmark runner records query ID, query name, dataset variant, storage format, layout, run ID, runtime, rows returned, stages, tasks, median runtime, best runtime, shuffle read/write, disk spill, broadcast-join usage, partition-pruning usage, and notes. This makes the results interpretable beyond raw runtime alone.

6.3 Metrics

The report focuses on the following metrics:

- **Median runtime:** robust comparison across repeated runs.
- **Shuffle volume:** amount of data moved across executors.
- **Disk spill:** amount of intermediate data written to disk due to memory pressure.
- **Plan evidence:** physical-plan signals such as `PartitionFilters`, `BroadcastHashJoin`, and `AdaptiveSparkPlan`.

7 Results and Discussion

7.1 Format Benchmark

Table 4 compares Parquet, ORC, and Avro. The workload includes projection scans, aggregation, route ranking, and origin–destination volume analysis.

Table 4: Format benchmark results: median runtime over measured runs.

Query	Parquet	ORC	Avro
Q01 projection scan	1.03s	0.68s	8.31s
Q03 vendor/zone aggregation	1.24s	1.06s	8.87s
Q06 route ranking	1.11s	0.77s	8.95s
Q07 origin–destination volume	0.53s	0.50s	8.45s

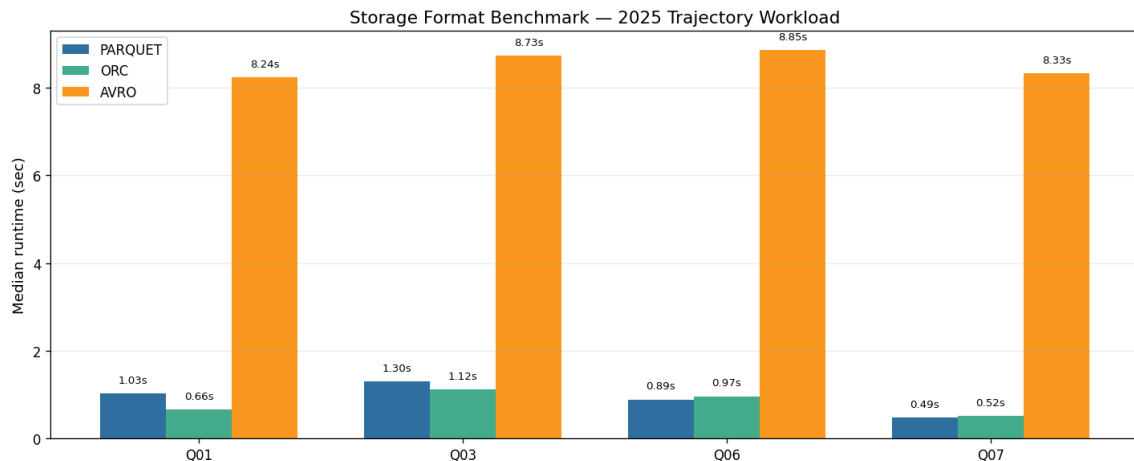


Figure 5: Format benchmark visualization. Parquet and ORC strongly outperform Avro on analytical trajectory queries.

Parquet is about $9.8\times$ faster than Avro, while ORC is about $12.3\times$ faster. The main reason is columnar storage. Q01 only needs a small subset of columns, so Parquet and ORC can perform column projection. Avro is row-oriented and must deserialize entire records. File size also supports the result: Parquet raw is approximately 1.04 GB, ORC raw is approximately 0.81 GB, and Avro raw is approximately 2.10 GB.

ORC is slightly faster than Parquet in this workload. This may come from ORC compression and indexing behavior. However, the difference between ORC and Parquet is much smaller than the difference between columnar formats and Avro. Therefore, the first recommendation is to avoid row-oriented formats for scan-heavy analytical trajectory workloads.

7.2 Partition Benchmark

Table 5 compares raw Parquet against year/month partitioned Parquet.

Table 5: Partition benchmark results: raw layout versus year/month partitioned layout.

Query	Raw	Partitioned	Speedup
Q02 time filter	0.52s	0.36s	30%
Q05 daily zone flow	0.65s	0.46s	29%
Q06 route ranking	0.88s	0.91s	-3%
Q07 origin–destination volume	0.49s	0.39s	20%

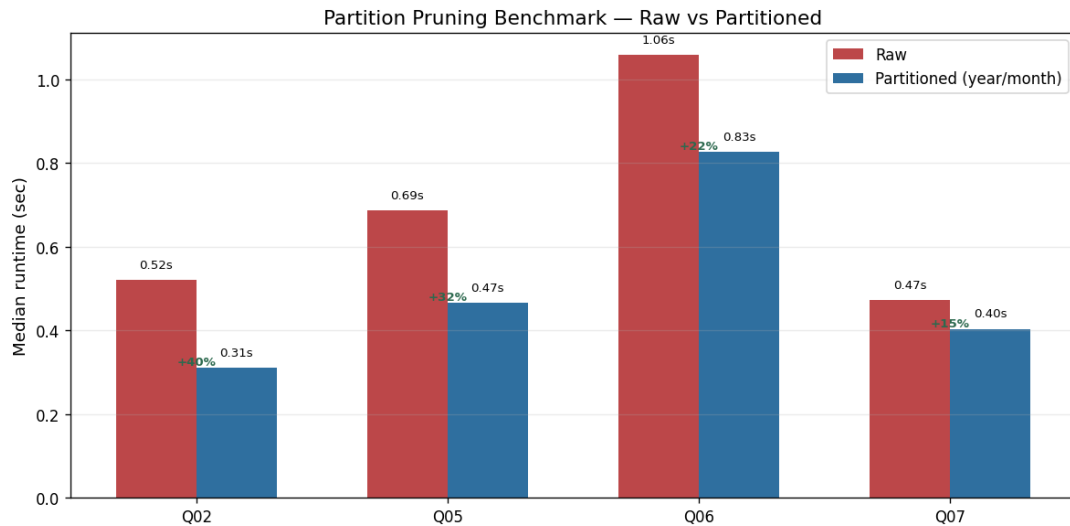


Figure 6: Partition benchmark visualization. Partition pruning improves most time-filtered queries.

Partition pruning works as expected. The physical plan confirms `PartitionFilters` on `trip_year = 2025` and `trip_month = 1`. Q02, Q05, and Q07 improve by 20–30% because Spark skips irrelevant partition directories before scanning files.

Q06 is a useful counterexample. It does not improve and is slightly slower on the partitioned layout. Q06 contains route aggregation and a `DENSE_RANK()` window function. The runtime is dominated by shuffle, grouping, sorting, and ranking rather than by scan cost. This shows that partitioning is effective only when filter-based data skipping is a major part of the workload.

7.3 Hash-bucketed Layout Benchmark

Table 6 compares raw Parquet with a 16-bucket hash layout by `origin_zone_id`.

Table 6: Hash-bucketed layout benchmark for zone-targeted queries.

Query	Raw	Bucketed	Speedup
Q11 zone lookup	1.32s	0.76s	43%
Q12 zone join	0.97s	0.55s	44%

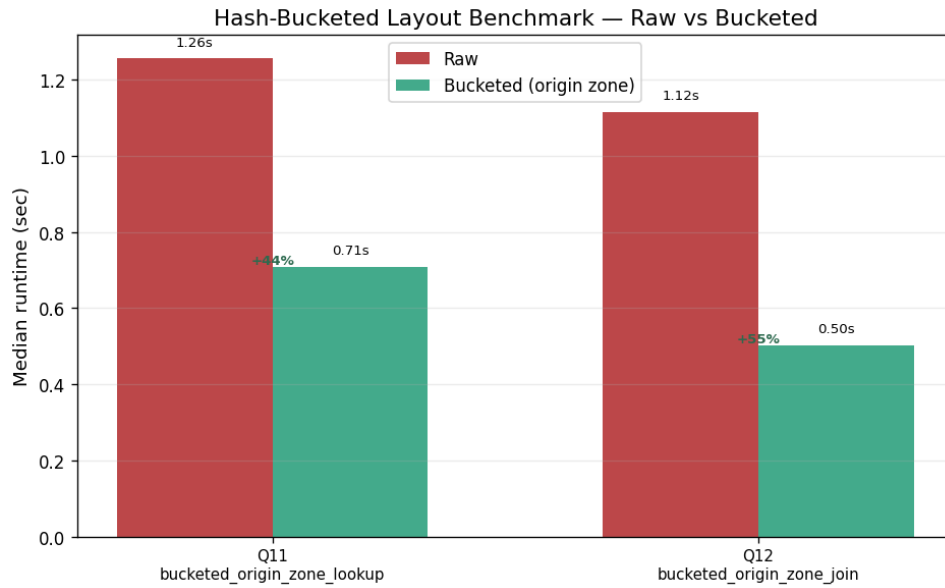


Figure 7: Hash-bucketed layout benchmark. Bucket-level data skipping accelerates zone-targeted lookup and join queries.

The bucketed layout improves both zone-targeted queries by approximately 43–44%. In theory, a 16-bucket layout can reduce scan scope to about one-sixteenth of the data for a single-zone query. The measured speedup is smaller because Spark still pays overhead for task scheduling, metadata listing, filtering, aggregation, and joining. Nevertheless, the result confirms that spatial bucketing is valuable when workloads repeatedly target specific origin zones.

7.4 Join and Skew Benchmark

Table 7 compares join strategies using the 265-row `taxi_zone_dim` table.

Table 7: Join/skew benchmark results.

Query	Strategy	Runtime	Shuffle	Spill
Q04	Skew aggregation	1.25s	0.5 MB	0 MB
Q08	SortMerge join	5.27s	25.9 MB	743 MB
Q09	Broadcast join	1.15s	0.9 MB	0 MB
Q10	Salted join	4.02s	0.9 MB	0 MB

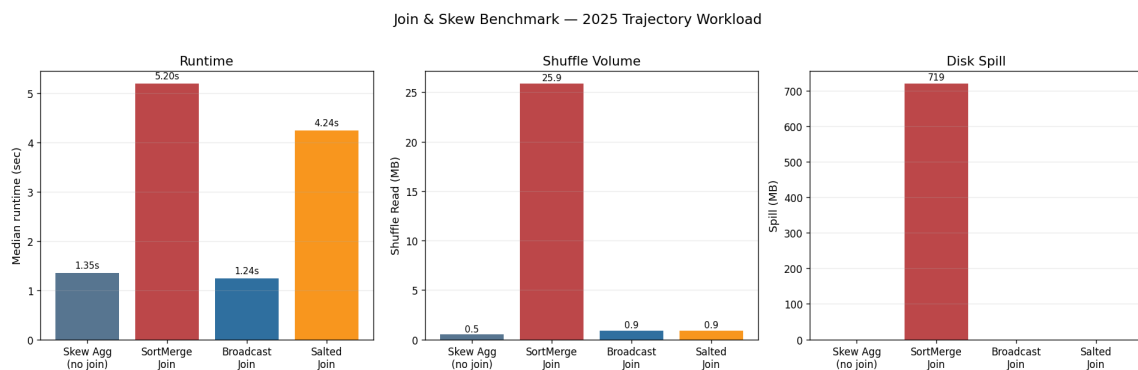


Figure 8: Join strategy comparison. Broadcast join is fastest because the dimension table is small.

Broadcast join is the best strategy in this benchmark. It reduces runtime from 5.27s to 1.15s, a $4.6\times$ improvement over SortMerge join. It also eliminates disk spill. The reason is that `taxi_zone_dim` has only 265 rows, so Spark can copy it to all executors and join locally with the large trajectory table.

SortMerge join performs poorly because it shuffles both inputs by `origin_zone_id` and sorts partitions before merging. Even with a small dimension table, the large fact table still drives shuffle and spill. The recorded 743 MB disk spill indicates that sorting intermediate data creates memory pressure.

Salted join reduces shuffle and avoids spill, but it is still slower than broadcast join. Salting is useful when both sides are large and a key is severely skewed. In this project, the dimension table is tiny, so salting adds unnecessary overhead by expanding the dimension across salt values and using a wider join key.

7.5 Adaptive Query Execution

Spark 4.0 enables Adaptive Query Execution by default in the project. The physical plans contain `AdaptiveSparkPlan`, confirming that AQE is active. AQE can coalesce shuffle partitions, convert eligible SortMerge joins into broadcast joins, and split skewed partitions at runtime. This makes the benchmark closer to realistic Spark behavior. However, because AQE is always enabled, the project does not isolate the independent contribution of AQE.

7.6 Overall Discussion

The results lead to four practical lessons:

1. **Use columnar formats for analytics.** Parquet and ORC strongly outperform Avro because analytical queries read selected columns across many rows.
2. **Partition by common filter columns.** Year/month partitioning improves time-window queries, but it does not help when shuffle and window computation dominate runtime.
3. **Bucket by common spatial keys.** Hash bucketing by origin zone is effective for repeated zone-targeted lookup and join workloads.

4. **Choose join strategy based on table size and skew.** Broadcast join is optimal for small dimensions, while salted join is only useful when broadcast is impossible and skew is severe.

These conclusions match the broader principle of big-data query optimization: the best layout depends on the access pattern. There is no universally optimal storage design. Instead, engineers must understand the dataset, query workload, and execution engine.

8 Limitations

The project is complete enough to demonstrate key optimization principles, but it has several limitations:

- **Single dataset and year.** The benchmark uses NYC Yellow Taxi data from 2025. Other datasets may have different schema width, skew, or temporal patterns.
- **Moderate scale.** The benchmark contains about 48.7 million rows and roughly 1 GB in raw Parquet. Petabyte-scale data may amplify the benefits of data skipping.
- **Single-node benchmark.** The documented benchmark uses four Spark workers on one physical node. Multi-node network and remote shuffle behavior may change the cost of joins.
- **AQE not isolated.** Adaptive Query Execution is enabled for all benchmark runs, so AQE-on versus AQE-off effects are not separately measured.
- **Limited layouts.** The project evaluates year/month partitioning and 16-bucket origin-zone hashing, but not Z-ordering, Bloom filters, Delta Lake, Iceberg, Hudi, or multi-column partitioning.
- **Data cleaning is not the focus.** The benchmark retains some anomalous trips because the primary goal is query optimization rather than statistical modeling.

9 Conclusion

This report analyzed the project *Distributed Query Optimization on Columnar Storage* for the Big Data and Processing course. The project builds an end-to-end Spark and MinIO pipeline for large-scale taxi trajectory processing. It normalizes NYC Yellow Taxi trips into a common `trips_clean` schema, prepares multiple storage layouts, executes Spark SQL benchmark queries, and evaluates runtime, shuffle, spill, and physical-plan evidence.

The experimental results show that physical data organization has a major impact on distributed query performance. Parquet and ORC are about 10–12× faster than Avro for analytical queries. Year/month partitioning improves time-filtered queries by 20–30%. Hash bucketing by origin zone improves zone-targeted queries by about 43–44%. Broadcast join is the best strategy for the small taxi-zone dimension, reducing runtime by 4.6× compared with SortMerge join and eliminating disk spill.

The main lesson is that scalable big-data processing requires workload-aware design. More CPU cores alone do not guarantee fast queries. Engineers must select file formats,

partition columns, bucketing keys, and join strategies according to the query workload and data distribution. Future work should evaluate the system on larger multi-node clusters, compare AQE enabled and disabled, test more advanced table formats, and explore additional indexing strategies for spatial trajectory analytics.

References

- [1] Dean, J., Ghemawat, S. (2004). MapReduce: Simplified Data Processing on Large Clusters. In: *OSDI*.
- [2] Zaharia, M., Chowdhury, M., Franklin, M. J., Shenker, S., Stoica, I. (2010). Spark: Cluster Computing with Working Sets. In: *HotCloud*.
- [3] Zaharia, M., Chowdhury, M., Das, T., Dave, A., Ma, J., McCauley, M., Franklin, M. J., Shenker, S., Stoica, I. (2012). Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing. In: *NSDI*.
- [4] Armbrust, M., Xin, R. S., Lian, C., Huai, Y., Liu, D., Bradley, J. K., Meng, X., Kaftan, T., Franklin, M. J., Ghodsi, A., Zaharia, M. (2015). Spark SQL: Relational Data Processing in Spark. In: *SIGMOD*.
- [5] Stonebraker, M., Abadi, D. J., Batkin, A., Chen, X., Cherniack, M., Ferreira, M., Lau, E., Lin, A., Madden, S., O'Neil, E., O'Neil, P., Rasin, A., Tran, N., Zdonik, S. (2005). C-Store: A Column-oriented DBMS. In: *VLDB*.
- [6] Abadi, D. J., Madden, S. R., Hachem, N. (2008). Column-Stores vs. Row-Stores: How Different Are They Really? In: *SIGMOD*.
- [7] Apache Software Foundation. Apache Parquet Documentation. <https://parquet.apache.org/>
- [8] Apache Software Foundation. Apache ORC Documentation. <https://orc.apache.org/>
- [9] Apache Software Foundation. Apache Avro Documentation. <https://avro.apache.org/>
- [10] Apache Software Foundation. Apache Spark SQL, DataFrames and Datasets Guide. <https://spark.apache.org/docs/latest/sql-programming-guide.html>
- [11] New York City Taxi and Limousine Commission. TLC Trip Record Data. <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>